```
import numpy as np

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

from sklearn.model_selection import train_test_split
```

```
sentences = [
    "book a flight from mumbai to delhi",
    "book a ticket from nagpur to pune",
    "find a flight from delhi to bangalore",
    "travel from mumbai to chennai",
    "flight from pune to hyderabad",
    "book flight from chennai to mumbai",
    "i want to fly from nagpur to delhi",
    "find ticket from bangalore to mumbai",
    "book a flight from hyderabad to pune",
    "travel from delhi to chennai",
    "i need a flight from pune to delhi",
    "book ticket from mumbai to nagpur",
    "find flight from chennai to bangalore",
    "travel from hyderabad to mumbai",
    "book a flight from delhi to pune",
    "i want to travel from nagpur to mumbai",
    "find a ticket from pune to chennai",
    "book flight from bangalore to delhi",
    "travel from mumbai to hyderabad",
    "find flight from chennai to pune"
]
```

```
labels = [
    "O O O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O B-from_city O B-to_city",
    "O O O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O O O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city"
]
```

```
# convert words into numbers.

tokenizer = Tokenizer()

tokenizer.fit_on_texts(sentences)

X = tokenizer.texts_to_sequences(sentences)
```

```
label_to_id = {
    "O": 0,
    "B-from_city": 1,
    "B-to_city": 2
}
```

```
y = []

for label_sentence in labels:

    label_ids = [
        label_to_id[label]
```

```
            for label in label_sentence.split()
        ]

        y.append(label_ids)
```

```
max_len = max(len(seq) for seq in X)
```

```
X = pad_sequences(
    X,
    maxlen=max_len,
    padding='post'
)
```

```
y = pad_sequences(
    y,
    maxlen=max_len,
    padding='post',
    value=0
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# LSTM model

vocab_size = len(tokenizer.word_index) + 1

num_labels = len(label_to_id)

model = Sequential()

model.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32,
        input_length=max_len
    )
)

model.add(
    LSTM(
        64,
        return_sequences=True
    )
)

model.add(
    Dense(
        num_labels,
        activation='softmax'
    )
)
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is depr
  warnings.warn(
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=4,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 72/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9583 - loss: 0.0890 - val_accuracy: 0.9688 - val_loss: 0.1796
Epoch 73/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9583 - loss: 0.0902 - val_accuracy: 0.9688 - val_loss: 0.1804
Epoch 74/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9583 - loss: 0.0880 - val_accuracy: 0.9688 - val_loss: 0.1874
Epoch 75/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9583 - loss: 0.0821 - val_accuracy: 0.9688 - val_loss: 0.1952
Epoch 76/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9583 - loss: 0.0797 - val_accuracy: 0.9688 - val_loss: 0.1957
Epoch 77/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9583 - loss: 0.0816 - val_accuracy: 0.9688 - val_loss: 0.1916
Epoch 78/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9583 - loss: 0.0763 - val_accuracy: 0.9688 - val_loss: 0.1852
Epoch 79/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9688 - loss: 0.0744 - val_accuracy: 0.9688 - val_loss: 0.1864
Epoch 80/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9583 - loss: 0.0770 - val_accuracy: 0.9688 - val_loss: 0.1917
Epoch 81/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9583 - loss: 0.0740 - val_accuracy: 0.9688 - val_loss: 0.1918
Epoch 82/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - accuracy: 0.9688 - loss: 0.0710 - val_accuracy: 0.9688 - val_loss: 0.1922
Epoch 83/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.9688 - loss: 0.0690 - val_accuracy: 0.9688 - val_loss: 0.1915
Epoch 84/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9479 - loss: 0.0706 - val_accuracy: 0.9688 - val_loss: 0.1927
Epoch 85/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9375 - loss: 0.0698 - val_accuracy: 0.9688 - val_loss: 0.1970
Epoch 86/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9375 - loss: 0.0665 - val_accuracy: 0.9688 - val_loss: 0.2017
Epoch 87/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9583 - loss: 0.0823 - val_accuracy: 0.9688 - val_loss: 0.2054
Epoch 88/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9583 - loss: 0.0685 - val_accuracy: 0.9688 - val_loss: 0.1899
Epoch 89/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9479 - loss: 0.0757 - val_accuracy: 0.9688 - val_loss: 0.1833
Epoch 90/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9479 - loss: 0.0725 - val_accuracy: 0.9688 - val_loss: 0.1878
Epoch 91/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9688 - loss: 0.0623 - val_accuracy: 0.9688 - val_loss: 0.1981
Epoch 92/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9583 - loss: 0.0638 - val_accuracy: 0.9688 - val_loss: 0.2032
Epoch 93/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9583 - loss: 0.0674 - val_accuracy: 0.9688 - val_loss: 0.2007
Epoch 94/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.9583 - loss: 0.0645 - val_accuracy: 0.9688 - val_loss: 0.1971
Epoch 95/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9688 - loss: 0.0597 - val_accuracy: 0.9688 - val_loss: 0.1925
Epoch 96/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9688 - loss: 0.0595 - val_accuracy: 0.9688 - val_loss: 0.1921
Epoch 97/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9583 - loss: 0.0610 - val_accuracy: 0.9688 - val_loss: 0.1915
Epoch 98/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9688 - loss: 0.0615 - val_accuracy: 0.9688 - val_loss: 0.1931
Epoch 99/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9583 - loss: 0.0584 - val_accuracy: 0.9688 - val_loss: 0.1972
Epoch 100/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.9792 - loss: 0.0558 - val_accuracy: 0.9688 - val_loss: 0.2043
```

```
loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("Test Accuracy:", accuracy)
```

```
Test Accuracy: 1.0
```

```
# Test a New Sentence
test_sentence = "book a flight from mumbai to pune"
```

```
test_sequence = tokenizer.texts_to_sequences(
    [test_sentence]
)
```

```
test_sequence = pad_sequences(
    test_sequence,
    maxlen=max_len,
    padding='post'
)
```

```
prediction = model.predict(
    test_sequence
```

```
)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 164ms/step
```

```
predicted_ids = np.argmax(
    prediction,
    axis=-1
)[0]
```

```
id_to_label = {
    0: "O",
    1: "B-from_city",
    2: "B-to_city"
}
```

```
words = test_sentence.split()
```

```
for word, label_id in zip(
    words,
    predicted_ids
):

    print(
        word,
        "->",
        id_to_label[label_id]
    )
```

```
book -> O
a -> O
flight -> O
from -> O
mumbai -> B-from_city
to -> O
pune -> B-to_city
```

```
import numpy as np

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Dense

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

from sklearn.model_selection import train_test_split


# ---------------------------------------------------
# 1. Create Dataset
# ---------------------------------------------------

sentences = [
    "book a flight from mumbai to delhi",
    "book a ticket from nagpur to pune",
    "find a flight from delhi to bangalore",
    "travel from mumbai to chennai",
    "flight from pune to hyderabad",
    "book flight from chennai to mumbai",
    "i want to fly from nagpur to delhi",
    "find ticket from bangalore to mumbai",
    "book a flight from hyderabad to pune",
    "travel from delhi to chennai",
    "i need a flight from pune to delhi",
    "book ticket from mumbai to nagpur",
    "find flight from chennai to bangalore",
    "travel from hyderabad to mumbai",
    "book a flight from delhi to pune",
    "i want to travel from nagpur to mumbai",
    "find a ticket from pune to chennai",
    "book flight from bangalore to delhi",
    "travel from mumbai to hyderabad",
    "find flight from chennai to pune"
]


# ---------------------------------------------------
# 2. Slot Labels
# ---------------------------------------------------
```

```
labels = [
    "O O O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O B-from_city O B-to_city",
    "O O O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O O O O B-from_city O B-to_city",
    "O O O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city",
    "O O B-from_city O B-to_city",
    "O O O B-from_city O B-to_city"
]


# ------------------------------------------------
# 3. Tokenization
# ------------------------------------------------

tokenizer = Tokenizer()

tokenizer.fit_on_texts(sentences)

X = tokenizer.texts_to_sequences(sentences)


# ------------------------------------------------
# 4. Encode Labels
# ------------------------------------------------

label_to_id = {
    "O": 0,
    "B-from_city": 1,
    "B-to_city": 2
}

y = []

for label_sentence in labels:

    label_ids = [
        label_to_id[label]
        for label in label_sentence.split()
    ]

    y.append(label_ids)


# ------------------------------------------------
# 5. Padding
# ------------------------------------------------

max_len = max(
    len(seq)
    for seq in X
)

X = pad_sequences(
    X,
    maxlen=max_len,
    padding='post'
)

y = pad_sequences(
    y,
    maxlen=max_len,
    padding='post',
    value=0
)


# ------------------------------------------------
# 6. Train-Test Split
```

```
# ------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)


# ------------------------------------------------
# 7. Build LSTM Model
# ------------------------------------------------

vocab_size = len(
    tokenizer.word_index
) + 1

num_labels = len(
    label_to_id
)

model = Sequential()

model.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32,
        input_length=max_len
    )
)

model.add(
    LSTM(
        64,
        return_sequences=True
    )
)

model.add(
    Dense(
        num_labels,
        activation='softmax'
    )
)


# ------------------------------------------------
# 8. Compile Model
# ------------------------------------------------

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)


# ------------------------------------------------
# 9. Display Model
# ------------------------------------------------

model.summary()


# ------------------------------------------------
# 10. Train Model
# ------------------------------------------------

history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=4,
    validation_split=0.2,
    verbose=1
)


# ------------------------------------------------
# 11. Evaluate Model
# ------------------------------------------------
```

```
loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print(
    "\nTest Accuracy:",
    accuracy
)


# --------------------------------------------------
# 12. Test New Sentence
# --------------------------------------------------

test_sentence = (
    "book a flight from mumbai to pune"
)

test_sequence = tokenizer.texts_to_sequences(
    [test_sentence]
)

test_sequence = pad_sequences(
    test_sequence,
    maxlen=max_len,
    padding='post'
)


# --------------------------------------------------
# 13. Prediction
# --------------------------------------------------

prediction = model.predict(
    test_sequence,
    verbose=0
)

predicted_ids = np.argmax(
    prediction,
    axis=-1
)[0]


# --------------------------------------------------
# 14. Convert IDs to Labels
# --------------------------------------------------

id_to_label = {
    0: "O",
    1: "B-from_city",
    2: "B-to_city"
}

words = test_sentence.split()

print("\nSlot Filling Result:")

for word, label_id in zip(
    words,
    predicted_ids
):

    print(
        word,
        "->",
        id_to_label[label_id]
    )
```